

2027 Gubernatorial Election Sentiment Analysis

A Full-Stack AI Platform for Real-Time Political Perception & Social Media Monitoring

What is this project?

This platform is an automated, production-grade social intelligence system developed for the **2027 Nigerian Gubernatorial Elections** (tracking pivotal states including Lagos, Rivers, Kano, Oyo, and Kaduna). It aggregates political chatter and news commentary, classifies sentiment into **Positive, Negative, or Neutral** using dual NLP engines, matches mentions to candidate entities, and presents live perception metrics (e.g. **Net Sentiment Index**) via an interactive editorial dashboard.

4 Platforms	Dual NLP	-1.0 to +1.0	12 / 12
X, FB, YouTube, News	VADER + DistilBERT	Compound Polarity Range	Automated Tests Passing

1. WHAT WE HAVE HERE — CORE SYSTEM MODULES

Multi-Source Ingestion Engine (<code>collectors.py</code>) • Live News Feeds: Continuous ingestion from Google News RSS, Vanguard, Daily Post, and Punch politics portals. • Social Platform Simulators: Ingestion pipeline for X (Twitter), Facebook, and YouTube election debates. • Automated Entity Recognition: Scans text for candidate aliases (e.g. Sanwo-Olu, GRV, Folarin) and links to state races.	Dual-Engine NLP Pipeline (<code>sentiment_engine.py</code>) • NLTK VADER: Fast lexical analysis tuned for social jargon, slang, emojis, caps, and exclamation marks. • DistilBERT Transformer: Deep bidirectional context model (<code>distilbert-base-uncased-sst-2</code>). • Graceful Fallback: Automatically falls back to VADER if GPU or transformer dependencies are unavailable.
Candidate Analytics & Leaderboards (<code>analytics.py</code>) • Net Sentiment Index: Normalizes candidate favorability as % Pos - % Neg (-100 to +100). • Rankings: Identifies <i>Most Favorable</i>, <i>Most Criticized</i>, and <i>Highest Social Buzz</i> candidates. • Candidate Detail Pages: Dedicated profiles with sentiment breakdowns, engagement metrics, and individual post feeds.	Interactive UI & NLP Sandbox (<code>templates/dashboard.html</code>) • Editorial Theme: Tactile paper styling (#FAF9F6) with Chart.js time-series area charts. • Dynamic Word Clouds: Rendered dynamically with ink palettes; filterable by Positive (praise) or Negative (critique) words. • Live NLP Sandbox: Interactive tool to test any custom quote or speech with immediate polarity feedback.

2. TECHNICAL STACK & ARCHITECTURAL ROLES

Architectural Layer	Technologies	Key Role & Implementation Responsibility
Application Server	Python 3.11, Django 4.2	MVC architecture, ORM queries, AJAX REST endpoints, session handling, and background task orchestration.
NLP & Intelligence	NLTK VADER, PyTorch, Hugging Face	Dual-engine scoring pipeline. Fast rule-based heuristics + deep contextual semantic evaluation.
Data Storage	SQLite3 / PostgreSQL	Relational schema with composite indexes on (platform, sentiment_label) and candidate relations.
Visualization & Word Clouds	Chart.js, WordCloud, Matplotlib	Time-series stacked area charts, platform comparison bars, and base64 server-generated word clouds.
Frontend & Interaction	Tailwind CSS, FontAwesome, JS AJAX	Editorial tactile paper design, asynchronous filtering (state, candidate, date window), and modal post viewers.

3. HOW WE DID WHAT WE DID — THE 4-STAGE PIPELINE

Stage 1: Ingestion & Named Entity Recognition (NER)	The collector polls live RSS feeds (Google News, Punch, Vanguard, Daily Post) and simulated social streams. Each item is passed to <code>match_candidate_and_race()</code> , which parses aliases and party keywords to automatically bind posts to the correct candidate and their 2027 state race.
Stage 2: Text Preprocessing & Dual NLP Scoring	Raw text is sanitized (stripping tracking URLs and HTML entities while preserving sentiment signals such as exclamation marks, capitalization, and emojis). It is evaluated via VADER or DistilBERT to output compound polarity (-1.0 to +1.0) and assign Positive , Neutral , or Negative labels.
Stage 3: Database Indexing & Analytics Aggregation	Scored posts are stored in Django models with composite indexes. The analytics engine leverages SQL aggregation (<code>Avg</code> , <code>Count</code> , <code>TruncDate</code>) to compute daily rolling trajectories and Net Sentiment Index figures dynamically without performance bottlenecks.
Stage 4: Responsive Visualization & Dynamic Filtering	Pre-aggregated datasets feed Chart.js visual charts. The word cloud module cleans election stopwords and builds custom ink-palette frequency maps. AJAX endpoints update the dashboard dynamically upon state, candidate, platform, or date filter selection.

4. RELATIONAL DATA MODEL & ENTITY RELATIONSHIPS

Model Entity	Key Attributes	Relationship & System Role
StateRace	state, year (2027), description, is_active	Top-level election contest. Groups candidates and posts geographically.
Candidate	name, party, alias_keywords, avatar_color, bio	Foreign key to StateRace. Stores aliases used by the NER matcher.
SocialPost	platform, content, sentiment_score, label, published_at	Foreign keys to Candidate and StateRace. Stores NLP scores & engagement.
CollectionJob	platform, posts_created, status, execution_time	Audit log tracking automated background scraping sessions and health.

5. KEY FORMULATIONS & ENGINEERING HIGHLIGHTS

Net Sentiment Index (NSI) Formula: To isolate genuine favorability from noise, the platform computes: NSI = (% Positive Posts) - (% Negative Posts) Yielding an index from -100.0 (universal disapproval) to +100.0 (universal acclaim) , independent of total discussion volume.	Lazy-Loading & Zero-Downtime Fallback: The NLP engine uses lazy imports for deep transformer models. If PyTorch or GPU resources are constrained, the pipeline falls back instantly to NLTK VADER—ensuring the dashboard and collectors remain fast and 100% operational.
--	---

6. VERIFICATION & HOW TO RUN

1. Environment & Database: <code>pip install -r requirements.txt && python manage.py migrate</code>
2. Seed Initial Election Data: <code>python manage.py seed_election_data --count 200</code> (seeds candidates, races, & posts)
3. Launch Dashboard: <code>python manage.py runserver</code> → Visit http://127.0.0.1:8000/ in browser.
4. Run Automated Tests: <code>python manage.py test</code> → Executes 12 comprehensive unit and integration tests.

Key Takeaway: The system transforms chaotic political chatter into rigorous, actionable perception intelligence—providing campaign teams, analysts, and citizens with transparent, data-driven election sentiment analytics.